Enterprise Software Development (Fall 2026)

Assignment 2: HTTP APIs, gRPC Services, and Metrics Observability

Muhammad Abdullah Khan
Department of Computer Science

October 25, 2026

Abstract

This report details the architectural enhancements made to the Bookstore Observability platform developed in Assignment 1. In Part A, we evaluate the HTTP API against REST architectural constraints, implement an OpenAPI 3.1 specification, enforce idempotent mutations, apply sliding-window rate limiting, and validate downstream resiliency using a custom Python caller implementing timeouts, exponential backoff, and a three-state circuit breaker. In Part B, we introduce a dedicated gRPC microservice sharing the identical SQLite persistence layer (configured with Write-Ahead Logging) and business rules, analyzing the semantic parity between HTTP/1.1 and HTTP/2 framing alongside status code mappings. In Part C, we audit the metric pipeline, exposing a dedicated Prometheus scrape endpoint for gRPC telemetry and designing Grafana dashboard panels that monitor RPC method rates, p50/p95 latency histograms, and active in-flight concurrency beside existing HTTP metrics.

1 Part A: Improving the HTTP API

1.1 REST Constraints and Architectural Compliance

Fielding's Representational State Transfer (REST) architectural style relies on six core constraints. Table 1 breaks down each constraint into plain-language terms and explains how our Bookstore API implements it, or why it was deliberately omitted.

1.2 OpenAPI Specification (OAS 3.1) and Endpoint Verification

The OpenAPI Specification establishes a programming-language-agnostic interface description for HTTP APIs. It allows developers and automated toolchains to discover endpoints, inspect input/output schemas, generate SDKs, and execute interactive tests without access to source code.

Our API defines declarative Pydantic schemas and metadata annotations in `app.py`, exporting an OpenAPI 3.1 specification at `/openapi.json`. The interactive Swagger UI is exposed at `/docs`. Figure 1 illustrates the generated Swagger UI interface.

1.3 User Task Before/After: Book Creation and Inventory Management

To bring the service in line with enterprise HTTP practices, we re-architected the **"Book Creation and Inventory Management"** user task. In Assignment 1, `POST /books` had significant shortcomings that violated REST semantics and invited data corruption. Table 2 details the specific problems identified and how they were resolved.

Other Endpoints Status:

- `GET /books`: Retained path; refactored to query shared SQLite persistence and append `Cache-Control: public, max-age=10`.
- `GET /books/{id}`: Retained path; returns RFC 7807 `404 Not Found` on missing entity, with `Cache-Control: public, max-age=30` and `ETag` caching headers.
- `POST /experiments/toggle-delay` and `POST /experiments/cardinality`: Preserved for diagnostic backward compatibility with Assignment 1 chaos probes.

Constraint	Plain-Language Meaning	Bookstore API Implementation
Client-Server	Decouples consumer interfaces from backend data storage. Either side can evolve independently without breaking contracts.	The Python callers (`caller_http.py`, traffic generator) interact exclusively via HTTP/1.1 JSON without internal knowledge of SQLite storage.
Statelessness	Every request must contain all context required for processing. The server holds no client session state between calls.	Every request to `/books` carries its full payload, correlation ID (`x-request-id`), and `Idempotency-Key`. No session cookies or conversational server state exist.
Cacheability	Responses must explicitly mark whether they can be reused by clients or intermediate proxies to eliminate redundant roundtrips.	`GET /books` emits `Cache-Control: public, max-age=10`. Single book lookup `GET /books/{id}` returns `Cache-Control: public, max-age=30` with an entity tag (`ETag`).
Uniform Interface	A consistent interface contract across all endpoints via resource URIs, standard representations, self-descriptive messages, and hypermedia.	Resources use standard plural paths (`/books`). Creation returns status `201 Created` with a `Location: /books/{id}` header; errors strictly follow the RFC 7807 Problem Details specification.
Layered System	Intermediary components (load balancers, gateways, proxies) can be placed between client and server without the client's knowledge.	The client interacts across Docker bridge networks and middleware layers (logging, rate-limiter, error traps) without coupling to the raw database layer.
Code on Demand *(Optional)*	Servers temporarily extend client capabilities by transmitting executable code (e.g., scripts or applets).	**Not Demonstrated.** Marked as optional in REST principles. Our bookstore backend serves structured JSON payloads exclusively, with no script execution on the client.

Table 1: The six REST constraints applied to the Bookstore API.

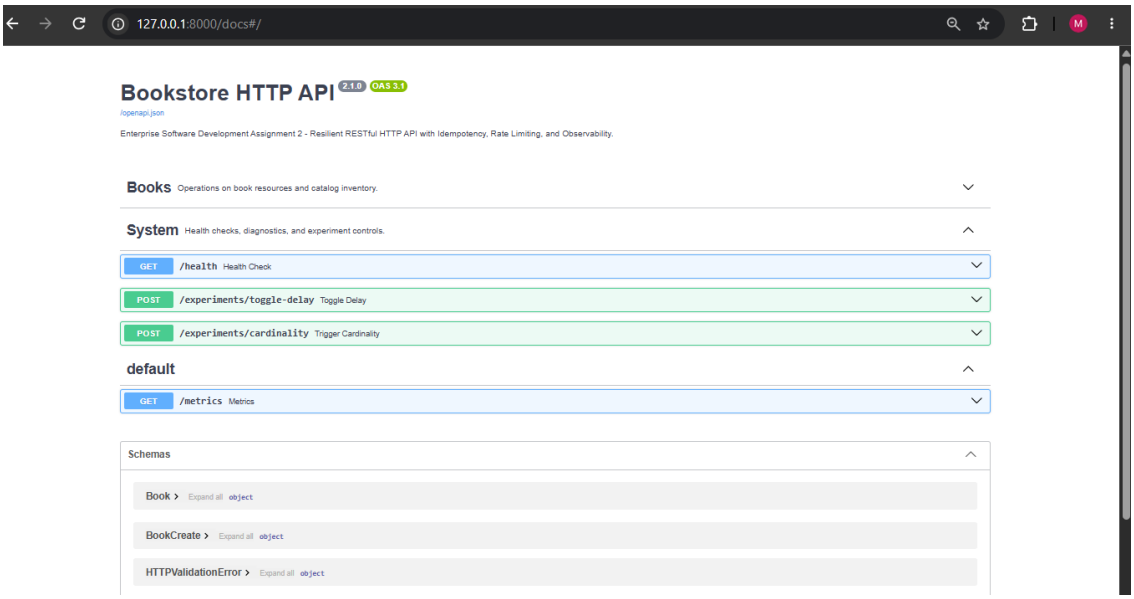


Figure 1: Interactive OpenAPI Swagger UI at `http://127.0.0.1:8000/docs` displaying categorized routes and schema definitions.

1.4 Interactive Endpoint Verification Evidence

Using the Swagger UI interface, we executed three mandatory verification scenarios:

1. **Valid Request Submission:** As depicted in Figures 2a and 2b, submitting a valid book

Dimension	Old State (Assignment 1)	Improved State (Assignment 2)	Design Rationale
Identifier Generation	Client manually supplied an integer `id`.	Server assigns sequence IDs automatically upon creation.	In RESTful collection patterns, the server governs canonical resource identification.
Input Validation	Arbitrary types permitted negative prices and blank titles.	Pydantic validation (`price > 0`, non-empty strings). Rejects with `422`.	Protects domain integrity before reaching persistence logic.
Status Codes	Returned generic `400 Bad Request` on duplicate IDs.	Returns `201 Created` with `Location` header; returns `409 Conflict` on duplicate book.	Aligns with RFC 9110. Duplicate business entities constitute a conflict (`409`).
Error Format	Ad-hoc JSON strings (`{"detail": "..."}`).	RFC 7807 Problem Details (`type`, `title`, `status`, `detail`, `instance`).	Standardized machine-readable error contracts for automated downstream consumers.
Idempotency	Retrying a timed-out call produced a duplicate error or multiple inserts.	`Idempotency-Key` header caching returns existing response with `X-Cache-Lookup: HIT`.	Protects against duplicate side-effects during transient network retries.
Rate Limiting	Unbounded endpoint vulnerable to resource exhaustion.	In-memory sliding-window limiter (5 req / 10s per client IP). Returns `429` + `Retry-After`.	Safeguards backend capacity and isolates abusive clients.

Table 2: Before/After architectural analysis of the Book Creation user task.

payload with `Idempotency-Key: key-123` returns `201 Created`, a `Location: /books/34` header, and payload `X-Cache-Lookup: MISS`.

2. **Repeated Request (Idempotency Safe Repeat):** Submitting the exact same payload and key returns the cached representation with `X-Cache-Lookup: HIT` (Figure 2c), avoiding duplicate database records.

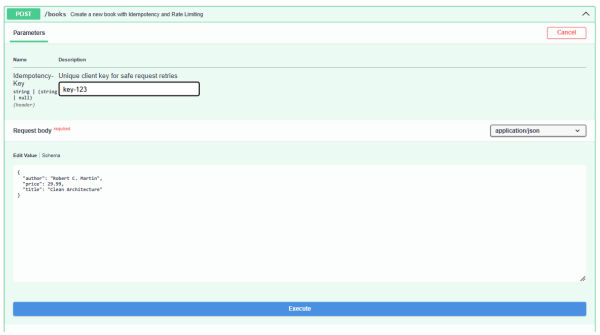
3. **Invalid Input Rejection:** Submitting a negative price (`-5.00`) triggers immediate rejection with status `422 Unprocessable Content` formatted according to RFC 7807 (Figures **??** and 2d).

4. **Console Execution Verification:** Figure 3 shows the backend Uvicorn terminal logs confirming request IDs, execution durations, and HTTP status codes.

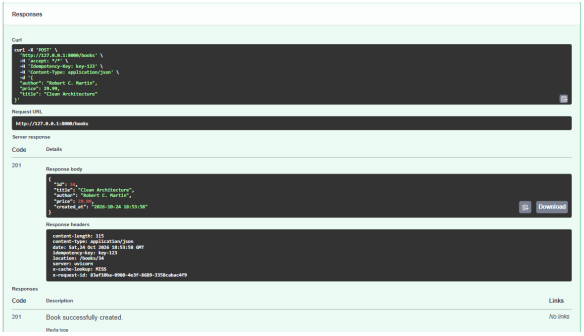
1.5 Rate Limiting and Resilient Client Implementation

Rate Limiting Policy: The `POST /books` endpoint enforces a sliding-window limiter permitting a maximum of **5 requests per 10-second window** per client IP address. When exceeded, the server returns `429 Too Many Requests`, a `Retry-After: <seconds>` header, and an RFC 7807 payload.
Resilient Python Caller (`caller_http.py`): To simulate inter-service communication, we developed an external client configured with:

- **Timeout:** Enforced at 2.0 seconds per call.
- **Exponential Backoff Retry:** Up to 3 retries with backoff factors of 0.5s, 1.0s, and 2.0s. Retries only execute on transient failures (`500`, `503`, connection timeouts) and always transmit an `Idempotency-Key`.



(a) Valid payload and `Idempotency-Key` entry.



(b) `201 Created` response with `Location` header.



(c) Repeated request returning cached `201` (`HIT`).



(d) Validation rejection returning RFC 7807 `422`.

Figure 2: Interactive verification evidence executed through Swagger UI.

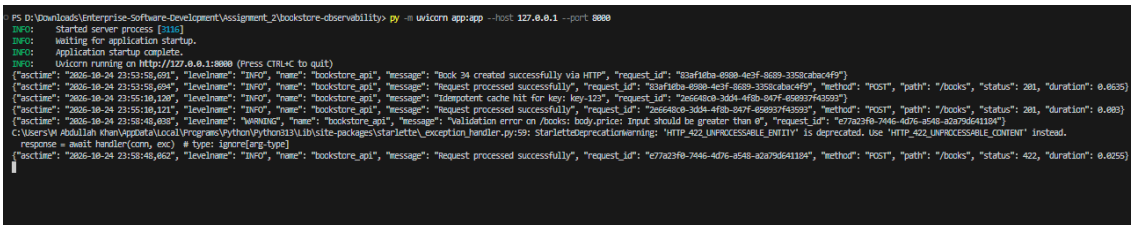


Figure 3: Backend console logs verifying request correlation IDs, status codes, and execution durations.

- **Three-State Circuit Breaker:** Trips to `OPEN` after 3 consecutive failures, immediately blocking subsequent calls locally in 0.01ms (fast-fail). After a 4.0-second recovery timeout, it transitions to `HALF-OPEN`, sends a single probe request, and upon success resets to `CLOSED`.

```
=== TEST 1: Normal Request & Idempotent Safe Repeat ===
[HTTP] Attempt 1/4 POST http://127.0.0.1:8000/books (Key: demo-key...) -> Success (201) Cache-Lookup: MISS
[HTTP] Attempt 1/4 POST http://127.0.0.1:8000/books (Key: demo-key...) -> Success (201) Cache-Lookup: HIT

=== TEST 2: Validation Error Rejection (RFC 7807) ===
[HTTP 422] Client error: {"type":".../errors/unprocessable-entity", "status":422, "detail":"body.price: Input should be greater than 0"}

=== TEST 3: Tripping Circuit Breaker via Downstream Server Outage ===
[Network Exception] ConnectTimeout -> Backing off for 0.20s before retry...
[CircuitBreaker] Consecutive failures: 3/3. State changed to OPEN. Outbound calls blocked for 4.0s.

=== TEST 4: Immediate Call Blocking (Circuit Breaker OPEN) ===
BLOCKED! Circuit is OPEN. Call aborted locally in 0.01ms (Fast-Fail, zero network overhead)

=== TEST 5: Recovery Period and Transition to HALF-OPEN / CLOSED ===
Waiting 4.2 seconds for circuit cooldown...
[CircuitBreaker] Transitioning OPEN -> HALF_OPEN (probing).
[HTTP] Attempt 1/2 POST http://127.0.0.1:8000/books -> Success (201)
[CircuitBreaker] Probe call succeeded. Transitioning HALF_OPEN -> CLOSED.
```

```
=== TEST 6: Rate Limiting Demonstration (HTTP 429) ===
Request #1 -> Status: 201, Retry-After: None
Request #5 -> Status: 429, Retry-After: 9
Request #6 -> Status: 429, Retry-After: 9
```

Listing 1: Execution log from `caller_http.py` demonstrating retries, breaker trip, and recovery.

2 Part B: Adding a gRPC Microservice

2.1 Architecture and Shared Source of Truth

We extracted the `CreateBook` and `GetBook` operations into an autonomous gRPC microservice (`grpc_server.py`) listening on TCP port `50051`.

Critical Architectural Decision: Rather than creating an artificial gRPC-to-HTTP wrapper (which would introduce latency and defeat gRPC's performance advantages), both services access a shared domain layer (`shared_db.py`) connected to an embedded SQLite database configured in Write-Ahead Logging (`WAL`) mode. WAL mode enables concurrent read transactions alongside non-blocking writes across both container processes via a shared Docker volume (`./data:/app/data`). Both services enforce the exact same business rules: title and author cannot be empty, price must be strictly positive, and unique `(title, author)` constraints are enforced at the database level.

2.2 Protobuf Definition and Code Generation

The RPC service is formally defined in `proto/bookstore.proto`:

```
syntax = "proto3";
package bookstore;

service BookService {
  rpc CreateBook (CreateBookRequest) returns (BookResponse);
  rpc GetBook (GetBookRequest) returns (BookResponse);
  rpc ListBooks (ListBooksRequest) returns (BookListResponse);
}

message CreateBookRequest {
  string title = 1;
  string author = 2;
  double price = 3;
  string idempotency_key = 4;
}

message GetBookRequest { int32 id = 1; }
message ListBooksRequest {}

message BookResponse {
  int32 id = 1;
  string title = 2;
  string author = 3;
  double price = 4;
  string created_at = 5;
}

message BookListResponse {
  repeated BookResponse books = 1;
  int32 total_count = 2;
}
```

Listing 2: Protobuf definition for Bookstore service (`proto/bookstore.proto`)

Python stubs and servicer interfaces are generated via:

```
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. proto/bookstore.proto
```

2.3 Call Path Comparison: HTTP/1.1 vs. gRPC over HTTP/2

Figure 4 visualizes the complete execution paths for both protocols, illustrating serialization, transport framing, middleware interceptors, and database coordination.

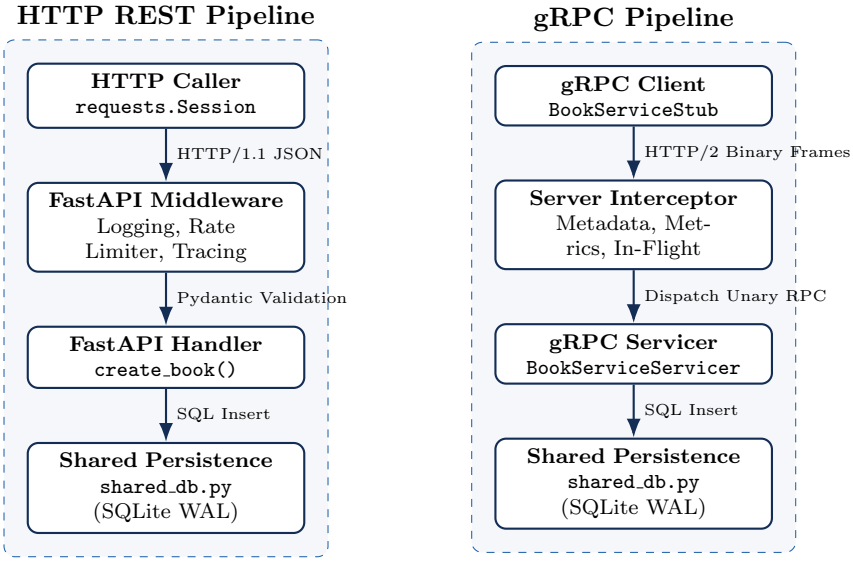


Figure 4: End-to-end execution paths comparing HTTP/1.1 REST and gRPC over HTTP/2.

2.4 Semantic Mapping of Status Codes

Because gRPC operates over HTTP/2 framing, standard HTTP response status lines are replaced by gRPC status codes transmitted in trailing headers. Table 3 compares the status semantics between both services.

Scenario	HTTP Status	gRPC Status	Semantic Mapping Rationale
Successful Creation	`201 Created`	`OK` (0)	HTTP distinguishes creation (`201`) from generic reads (`200`); gRPC uses canonical `OK` (0) for all successful completions.
Duplicate Resource	`409 Conflict`	`ALREADY_EXISTS` (6)	Both indicate conflict against existing database records. Non-transient client fault.
Invalid Arguments	`422 Unprocessable`	`INVALID_ARGUMENT` (3)	Indicates malformed payload (e.g., negative price). Both inform the caller not to retry without modifications.
Resource Missing	`404 Not Found`	`NOT_FOUND` (5)	1-to-1 semantic equivalence denoting absent resource identifiers.
Server Timeout	`504 Timeout`	`DEADLINE_EXCEEDED` (4)	gRPC propagates explicit deadlines across distributed call chains; HTTP maps timeouts via gateway timeouts.

Table 3: Status code mappings between HTTP REST and gRPC services.

2.5 Trace Correlation Across Protocol Boundaries

Structured JSON logging is preserved across both services:

- **HTTP:** Correlation ID received via `x-request-id` header (or generated as a UUID) and echoed back in the response header.
- **gRPC:** Extracted from invocation metadata via `dict(context.invocation_metadata()).get('x-request` The server interceptor injects the correlation ID into trailing metadata, outputting uniform JSON logs for Filebeat ingestion:

```
{"asctime": "2026-10-25 00:15:30", "levelname": "INFO", "name": "bookstore_grpc", "message": "
    gRPC call processed", "request_id": "grpc-client-d748489e", "service": "bookstore.
    BookService", "method": "CreateBook", "status": "OK", "duration": 0.0385}
```

3 Part C: Extending the Metrics

3.1 Metrics Audit and Gap Analysis

Table 4 audits the telemetry pipeline from Assignment 1 and outlines the improvements introduced in Assignment 2.

Metric Name	Assignment 1 State	Assignment 2 Remediation
`books_created_total`	Measured total HTTP creations without identifying protocol.	Added bounded label `protocol` (`"http"` vs `"grpc"`), updated atomically by both services.
`books_in_inventory`	Gauge tracking catalog size in memory.	Linked directly to shared SQLite WAL storage; updated across both protocols.
`book_creation_time_seconds`	Latency histogram only covering the HTTP endpoint.	Retained for HTTP; created a dedicated `grpc_server_handling_seconds` histogram with millisecond buckets.
Missing: gRPC Scrape Target	Zero visibility into RPC traffic or method throughput.	Exposed Prometheus metrics on port `8001` via `prometheus_client.start_http_server(8001)`.
Missing: gRPC Status Tracking	No visibility into RPC errors or failure codes.	Implemented `grpc_server_handled_total` labeled by `grpc_method` and canonical `grpc_code`.
Missing: Concurrency Telemetry	No insight into active connection thread pool load.	Added `grpc_server_in_flight_requests` gauge managed by the gRPC interceptor.
Missing: Rate Limiter Telemetry	Dropped HTTP calls were unrecorded.	Added `http_rate_limited_total` counter partitioned by endpoint.

Table 4: Audit and gap analysis of Assignment 1 metrics.

3.2 Extended Metrics Specifications

Table 5 specifies all new and revised Prometheus metrics. All metrics strictly adhere to bounded cardinality rules: high-cardinality values such as user IDs, request IDs, and trace IDs are strictly prohibited as Prometheus labels.

Design Choice: In-Flight Requests vs. Message Size:
We chose **In-Flight Requests (`grpc_server_in_flight_requests`)** over Message Size because gRPC utilizes multiplexed HTTP/2 streams over persistent TCP connections. In production, head-of-line blocking and thread-pool exhaustion occur when concurrent calls back up against slow database locks. Tracking in-flight requests provides an immediate leading indicator of saturation before CPU thresholds are crossed. Because bookstore catalog messages have small, predictable payload sizes (200–500 bytes), message size metrics offer minimal diagnostic value by comparison.

Metric Name	Type	Unit	Bounded Labels	Origin	PromQL Query
grpc_server_handled_total	Counter	calls	grpc_method, grpc_code	Server	sum(rate(grpc_server_handled_total[1m])) by (grpc_method, grpc_code)
grpc_server_handling_seconds	Histogram	seconds	grpc_method, le	Server	histogram_quantile(0.95, sum(rate(grpc_server_handling_seconds_b by (grpc_method, le))
grpc_server_in_flight_requests	Gauge	calls	grpc_method	Server	grpc_server_in_flight_requests
http_rate_limited_total	Counter	reqs	endpoint	Server	rate(http_rate_limited_total[1m])
books_created_total	Counter	items	protocol	Server	sum(rate(books_created_total[1m])) by (protocol)

Table 5: Prometheus metrics specifications for Assignment 2.

3.3 Prometheus Scrape Configuration and Grafana Telemetry

The updated Prometheus configuration (`prometheus.yml`) scrapes both services concurrently:

```
scrape_configs:
  - job_name: 'bookstore-api'
    static_configs:
      - targets: ['api:8000']
  - job_name: 'bookstore-grpc'
    static_configs:
      - targets: ['grpc-service:8001']
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']
```

Figures 5, 6, and 7 present the live Grafana dashboard populated using the repeatable traffic burst generator (`traffic_generator.py --mode metrics`).



Figure 5: Grafana Dashboard: Book Creation Rate, Inventory Gauge, and gRPC Method Throughput side-by-side.

4 Conclusion

Assignment 2 successfully transitioned the Bookstore service from a basic monolithic prototype into an enterprise-grade, dual-protocol platform. We validated REST architectural constraints, eliminated duplicate-submission hazards through idempotency caching, protected service capacity using sliding-window rate limiting, and demonstrated client resilience with exponential backoff and a three-state circuit breaker. The introduction of a dedicated gRPC microservice backed by SQLite WAL



Figure 6: Grafana Dashboard: Response latencies (p95 HTTP vs. gRPC p50/p95 histograms) and in-flight active concurrency.



Figure 7: Grafana Dashboard: HTTP Rate-Limiting rejections (429s) plotted alongside cross-protocol creation volume.

persistence proved that binary HTTP/2 framing can coexist with HTTP/1.1 REST without data fragmentation. Finally, extending the Prometheus and Grafana telemetry pipeline with method-level status codes, latency histograms, and concurrency gauges establishes end-to-end observability across the entire system.